

SMART NURSERY GUARDIAN

An AI-Assisted Embedded Monitoring System for Infant Care

1. Introduction

Smart Nursery Guardian is an AI-assisted embedded monitoring system designed to help caregivers monitor an infant. The system listens to the baby's cry and uses machine learning to classify the possible cause as hungry, tired, or discomfort. It also uses sensors to monitor motion, light, temperature, and gas or smoke.

The main idea is to reduce guesswork and provide a quick response when the baby cries or when a safety problem is detected.

2. Project Objectives

- Detect the baby's voice or cry using a laptop microphone.
- Process the audio and extract useful audio features.
- Use a trained machine learning model to classify the cry.
- Start crib rocking when a cry is detected.
- Display the baby's possible condition on the GUI.
- Monitor motion, light, temperature, and gas using the microcontroller.
- Control the room light and fan according to sensor conditions.
- Send a Telegram alert when gas or smoke is detected.

3. System Components

The project is divided into software and hardware parts.

1 Software

- Python
- Tkinter for the graphical user interface.
- NumPy and Librosa for audio processing and feature extraction.
- Joblib for loading the trained machine learning model.
- PySerial for communication with the microcontroller.
- python-telegram-bot for Telegram notifications.

2 Hardware

- STM32 BlackPill or a suitable microcontroller.
- PIR motion sensor.
- Light sensor (LDR).
- Temperature sensor.
- Gas sensor.
- Servo motor for crib rocking.

- Buzzer.
- RGB LED and room LED.
- DC motor and fan with a motor driver.
- Breadboard and connecting components.

4. System Flow

The system has two main parts running together: the Python application on the laptop and the microcontroller. They communicate through serial communication when an important event occurs.

1 Startup

1. The microcontroller starts and initializes the sensors and actuators.
2. The Python application starts and loads the trained ML model.
3. The serial connection is opened.
4. The Tkinter GUI starts in the normal monitoring state.
5. The microphone starts listening for audio.

2 Audio and Cry Detection

6. The microphone captures audio in small chunks.
7. Background noise is reduced as part of the audio-processing pipeline.
8. A simple voice/cry detection step checks whether significant sound is present.
9. When a cry is detected, a cry-detected signal can be sent to the microcontroller.
10. The servo starts rocking the crib.
11. Audio features are extracted and passed to the trained ML model.
12. The predicted condition is displayed on the GUI.
13. When the cry ends, a cry-ended signal is sent and the rocking can stop.

3 Machine Learning Result

- Hungry: the GUI displays a message and can play a calming video.
- Tired: the GUI displays a warning and the buzzer can be activated.
- Discomfort: the GUI suggests checking the diaper, clothing, or baby's position.

4 Sensor and Safety Flow

14. The microcontroller continuously reads the PIR, light, temperature, and gas sensors.
15. The PIR sensor counts motion in an 8-second window. Four or more motions indicate that the baby is awake.
16. The room LED is turned on when the room is dark and the baby is awake or a cry is being detected.
17. Temperature controls the fan and the temperature status indication.
18. If gas or smoke exceeds the selected threshold, the buzzer is activated and an alert is sent to the laptop.
19. The Python application shows the safety alert and sends a Telegram message to the caregiver.

5. Communication Between Python and Microcontroller

The project uses serial communication only for sensor events and control commands. Audio is processed on the laptop and is not sent through the serial connection.

Examples of messages from the microcontroller to Python:

- Baby awake
- Gas or smoke alert
- Temperature reading

Examples of commands from Python to the microcontroller:

- Cry detected / Cry ended
- Servo start / Servo stop
- Buzzer on

6. Telegram Notification

A Telegram bot is used to notify the caregiver when gas or smoke is detected. The bot was created using BotFather, and the required bot token and chat ID were configured in the Python code. The python-telegram-bot library is used to send the warning message.

This method is simple because the notification can reach the caregiver's phone without requiring a separate server.

7. Work Completed

The following parts were implemented or prepared during the project:

- A Tkinter GUI was created for Smart Nursery Guardian.
- Audio monitoring was connected to the GUI with Start and Stop controls.
- The audio system calculates RMS and detects significant voice/cry activity.
- MFCC features are extracted from the recorded audio.
- The saved machine learning model can be loaded by the Python application.
- The GUI displays the predicted baby condition and confidence when available.
- The main sensor and actuator logic was prepared for the microcontroller.
- A Telegram bot was created and tested by sending a warning message.
- The Python application was prepared to communicate with the microcontroller through serial communication.
- A simulation mode can be used to test the GUI without having the real hardware connected.

8. Results

The project resulted in a working prototype structure that combines audio processing, machine learning, GUI monitoring, microcontroller sensors, actuators, and Telegram notification.

- The GUI can start and stop audio monitoring.
- The audio status can be displayed in real time.

- MFCC features can be extracted and sent to the saved ML model.
- The predicted baby condition can be shown on the GUI.
- The system design supports automatic crib rocking after cry detection.
- Temperature, gas, light, and motion readings can be handled by the microcontroller.
- Gas detection can generate a high-priority warning and Telegram notification.

9. Conclusion

Smart Nursery Guardian combines embedded systems, audio processing, machine learning, and IoT communication in one infant-monitoring project. The Python application handles audio processing, classification, GUI monitoring, and Telegram communication, while the microcontroller handles the sensors and actuators. The final system is designed to react quickly to crying and safety events and provide useful information to the caregiver

